

GAN Dashboard

DATS 6303: Final Project

Jeongwon Yoo, Pratik Shetty, Josh Schechter

Generative AI models have been increasingly improving over the years. For our final project, we sought a way to explore some of the backbone generative AI models, specifically for image generation. We selected to explore different variations of the Generative Adversarial Network (GAN) Architecture. We started with a simple DCGAN architecture, then worked on a WGAN-GP implementation, and finally attempted to train a ProGAN model. Once training was complete, we put together an interactive dashboard using streamlit, to display our results and demonstrate our understanding of the architectures. Our evaluation metric was Frechet Inception Distance (FID), which we selected because it both tracks diversity of its own images, and also the distance from a real image.

The dataset we used was a kaggle dataset with 140k real and fake images. This amounted to 70k real images for training our discriminator (will discuss this later).

<***finish this section later***>.

DCGAN was our baseline GAN architecture and the first model we implemented. Like a standard GAN, it consists of a generator and a discriminator trained against each other. The main difference is that DCGAN replaces fully connected image generation with convolutional design, making it more suitable for image data. In our implementation, the generator starts from a 100-dimensional latent noise vector and upsamples it into a 64×64 RGB face image using transposed convolution layers, Batch Normalization, and ReLU, with a final Tanh output. The discriminator reverses this process by taking a 64×64 RGB image and downsampling it through convolution layers with LeakyReLU and Batch Normalization until it outputs a real/fake probability. This

made DCGAN a strong baseline for understanding adversarial image generation before moving to WGAN-GP and ProGAN.

The WGAN architecture uses the earth mover distance for the critic rather than a probability distribution, used by DCGAN. The earth mover function outputs a raw scalar value that represents how far off the generated image is from a real one. Because this does not rely on direct overlap between the real and sampled distributions, this allows for a smoother training curve.

However, there is a caveat with the earth mover function. The critic must be 1-Lipschitz compliant. This means that the output of the function must not outgrow the input of the function by more than a linear relationship. The original solution to this was weight clipping, which basically just bounded the weights to ensure the outputs were smooth. However, this was not ideal, and led to poor results. This led to the introduction of a gradient penalty. This gradient penalty would penalize outputs that were too strong. This added more stability.

ProGAN takes a fundamentally different approach from DCGAN and WGAN-GP by growing the generator and discriminator progressively during training. Rather than training at the target resolution from the start, ProGAN begins at a very low resolution of 4×4 pixels and gradually doubles it (8×8 , 16×16 , 32×32) all the way up to the target resolution of 64×64 or 128×128 . This allows the model to first learn the global structure of a face before refining finer details like textures and hair.

A key challenge with this approach is how to introduce new layers without disrupting what the network has already learned. ProGAN addresses this with a fade-in mechanism, controlled by an alpha parameter that goes from 0 to 1. At alpha equals 0,

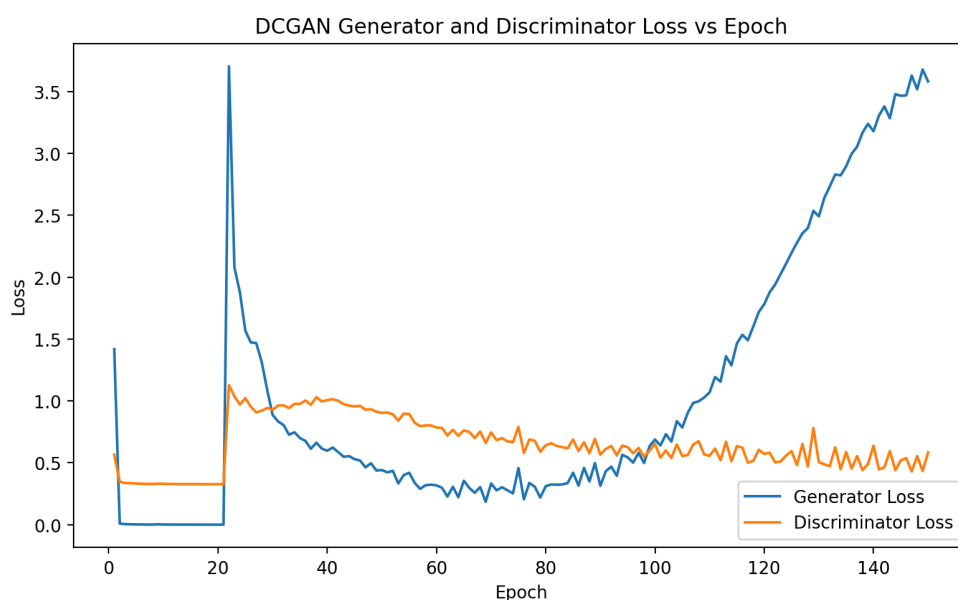
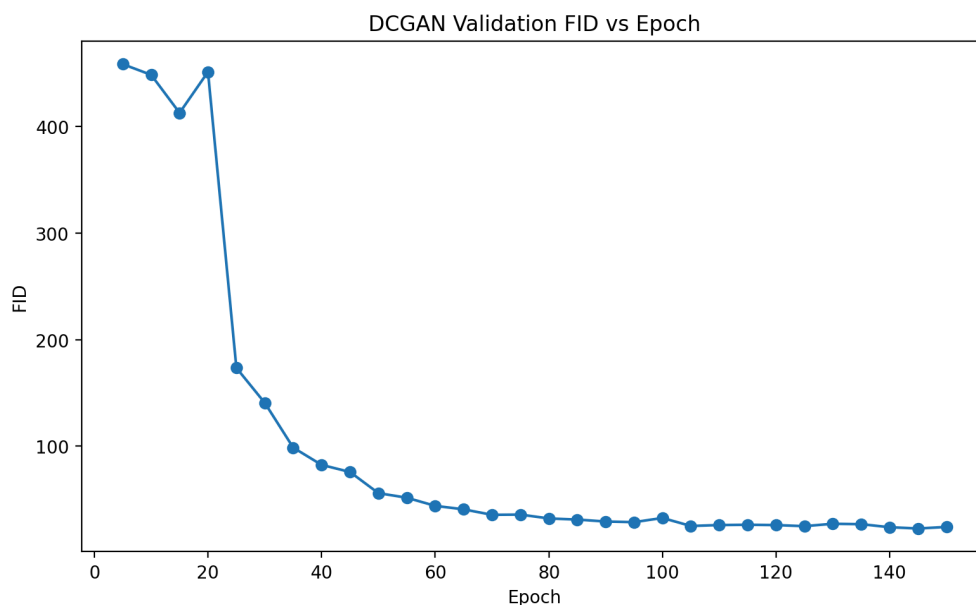
the network relies entirely on the previous resolution. As α increases, the new layer is gradually blended in until it fully takes over at α equals 1. This smooth transition prevents sudden shocks to the already-learned features.

ProGAN also introduces three additional techniques to further stabilize training. Equalized learning rate scales the weights dynamically at runtime, ensuring all layers train at the same effective speed. Pixelwise feature normalization prevents signal magnitudes from exploding in the generator. Finally, minibatch standard deviation adds diversity statistics to the discriminator, which discourages mode collapse — the tendency for the generator to produce repetitive, similar images. Like WGAN-GP, ProGAN uses the Wasserstein loss with a gradient penalty to enforce the Lipschitz constraint and maintain stable gradients throughout training.

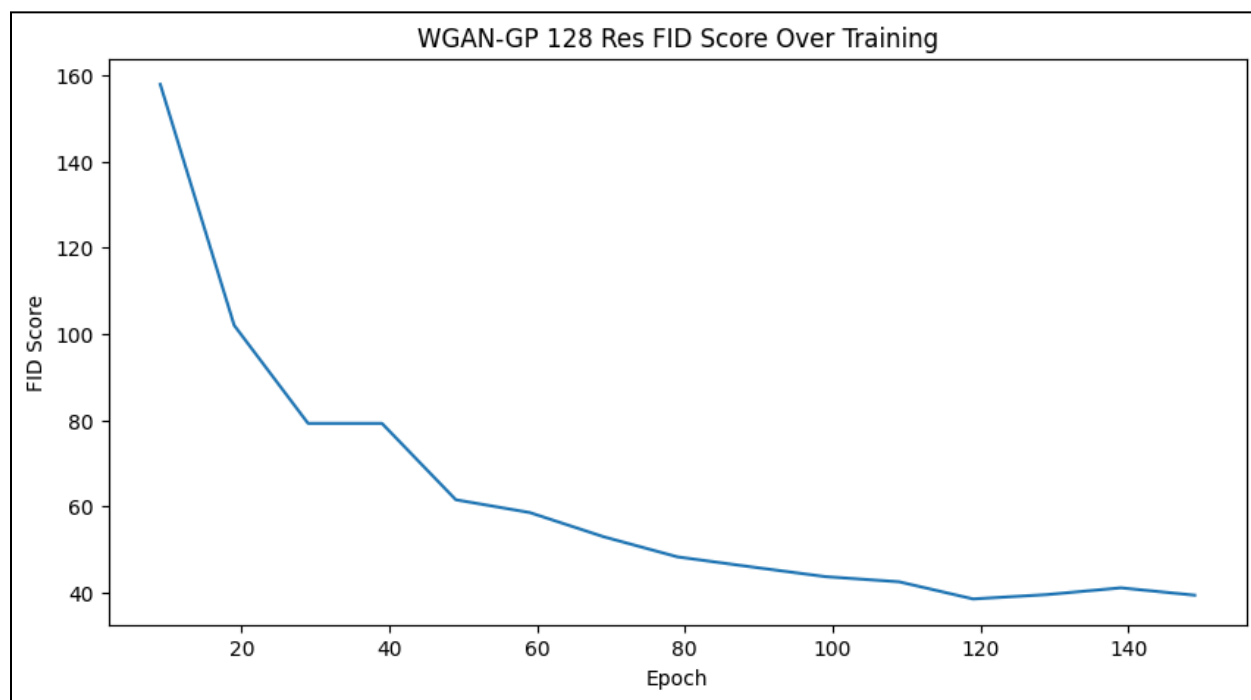
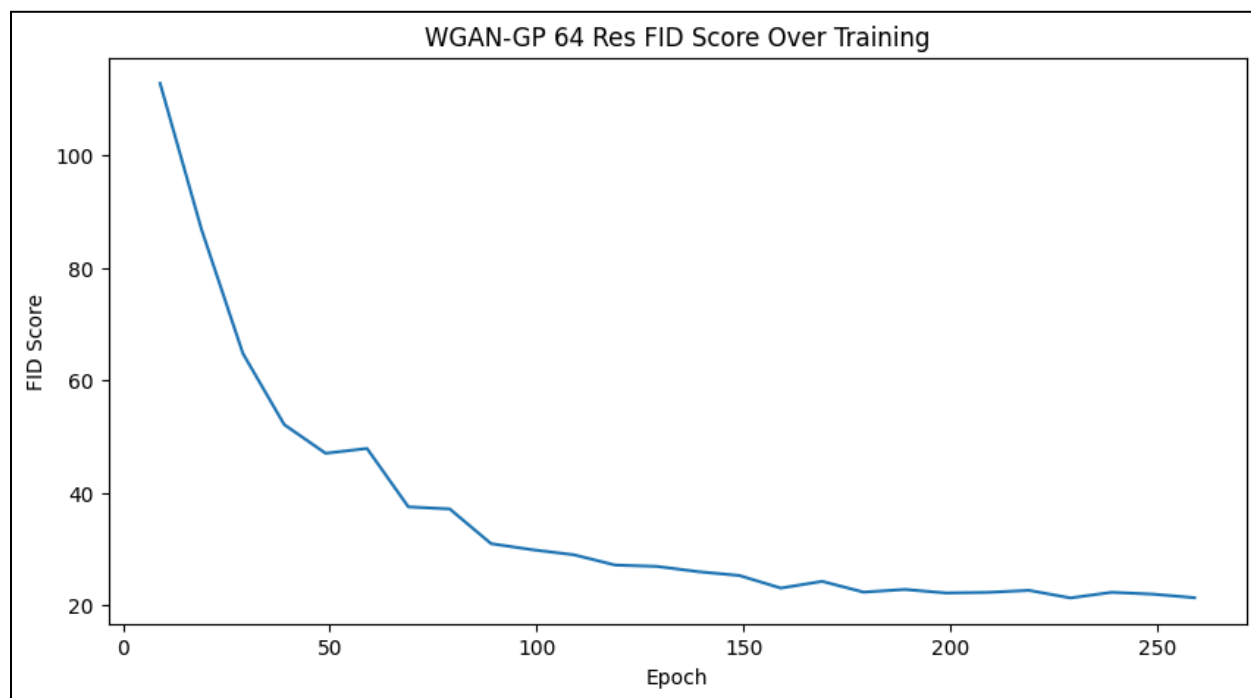
The next step was training. Each model was trained under the exact same preprocessing requirements.

For training, DCGAN was used only at 64×64 resolution and served as our baseline comparison model. We trained it with batch size 128, 150 epochs, and feature maps of 128. The generator and discriminator used separate Adam learning rates, $1e-4$ and $5e-5$ respectively, to reduce instability. Training used binary cross-entropy loss, with real-label smoothing at 0.9 and annealed instance noise early in training to keep the discriminator from becoming too strong too quickly. We evaluated the model using FID every five epochs and saved the checkpoint with the best validation FID, then reloaded that checkpoint at the end. DCGAN reached recognizable results quickly, but it was less stable than WGAN-GP, which is why it was kept mainly as a baseline and reference model in the final comparison.

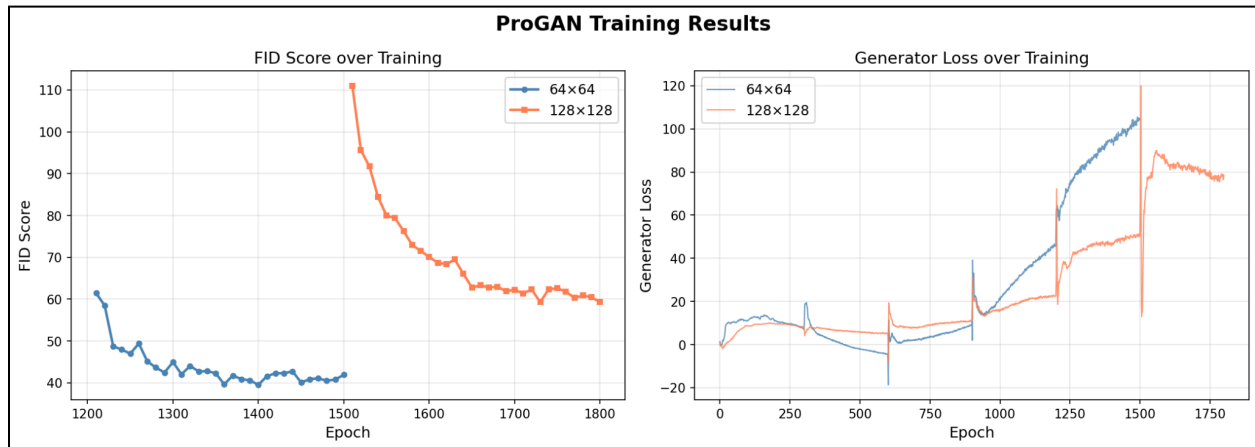
The following plots summarize DCGAN training behavior. The first plot shows validation FID over training epochs, where lower values indicate that the generated images are becoming closer to the real data distribution. The second plot shows generator and discriminator loss across epochs, which helps illustrate the adversarial learning behavior between the two networks during training.



Training the WGAN-GP model was relatively straightforward. We ran tuning on a large search space one time, then moved on to finer tuning and adjustments specific to each model. The 64 image sized outperformed the 128 image sized run.



Here are the result plots of ProGAN.

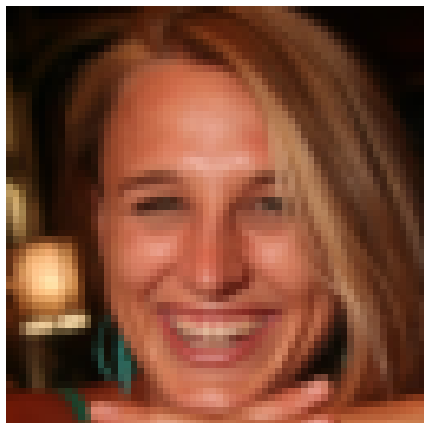


The left plot shows the FID score over training epochs for both resolutions. Both models demonstrate a consistent downward trend, indicating progressive improvement in image quality throughout training. The 64×64 model achieved a best FID of 39.5, while the 128×128 model reached a best FID of 59.3. The higher FID for the 128×128 model is expected, as generating higher resolution images is inherently more challenging and requires more training to converge.

The right plot shows the generator loss over training epochs. Both models exhibit an initial period of instability during the early progressive growing steps, followed by gradual stabilization as training progresses. The spikes visible in the loss curves correspond to transitions between resolution steps, where new layers are introduced via the fade-in mechanism.

<MODEL RESULT COMPARISONS HERE>

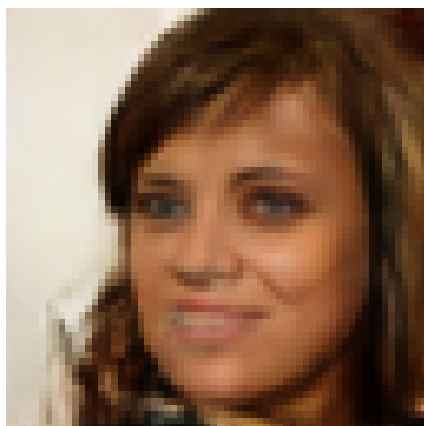
Real image example (64 pixels):



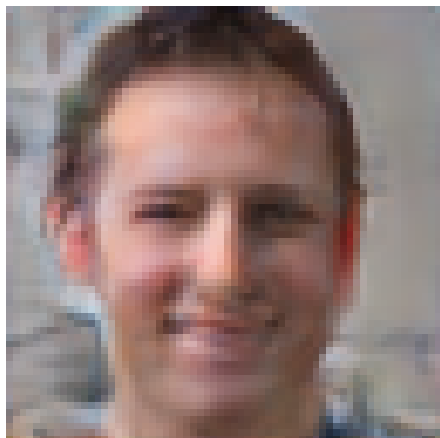
DCGAN- Image example (64 pixels)



WGAN-GP Image example (64 pixels):



ProGAN Image example (64 pixels):



ProGAN Image example (128 pixels):



<FINAL CONCLUSIONS HERE>

Per our study, we found that WGAN-GP ended up performing the best for our dataset. In our experiment it turned out that WGAN-GP ended up having the best all around results. This was expected over DCGAN, but it was surprising that it performed better than ProGAN, even at higher resolutions. A few theories could be had here. First, the image resolutions may not have been large enough for ProGAN to become beneficial enough. Second, ProGAN is extremely difficult to train and tune, especially

over a short period of time. If given more time, it is very possible that ProGAN would have outperformed at an image resolution of 128.

References

- Goodfellow et al. (2014), *Generative Adversarial Nets*
<https://arxiv.org/pdf/1406.2661>
- Radford et al. (2015), *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*
<https://arxiv.org/pdf/1511.06434>
- Arjovsky et al. (2017), *Wasserstein GAN*
<https://arxiv.org/abs/1701.07875>
- Gulrajani et al. (2017), *Improved Training of Wasserstein GANs*
<https://arxiv.org/abs/1704.00028>
- Karras et al. (2017), *Progressive Growing of GANs for Improved Quality, Stability, and Variation*
<https://arxiv.org/abs/1710.10196>
- Zeiler and Fergus (2010), *Deconvolutional Networks*
<https://www.matthewzeiler.com/mattzeiler/deconvolutionalnetworks.pdf>
- EmilienDupont, *wgan-gp* GitHub repository
Consulted for reference while implementing WGAN-GP ideas; code was not copied or directly reused.
<https://github.com/EmilienDupont/wgan-gp>